

SYSTEM DESIGN

Most Repeated Interview Questions

yashuintech

SDE Interview & Job Prep

yashudeveloper

AI & Vibe Coding

Overview

Category	# Questions
Scalability Basics	6
Databases	7
Caching	5
Distributed Systems	6
Real Product Designs	7

SCALABILITY BASICS

Q1. Scaling up vs scaling out? 📌 MOST REPEATED

Scaling up means making your single server more powerful — more RAM, more CPU, faster disks. It's simple to do because you don't need to change how your app is built, but every machine has a maximum size it can grow to, and if that one machine goes down, your whole system goes down with it. Scaling out means adding more machines and splitting the work between them. This is harder to set up, because your app now has to work across multiple servers instead of relying on just one, but it lets you handle far more traffic and gives you backup machines if one fails. Most large systems eventually scale out, simply because there's a hard ceiling on how big one single machine can get.

Q2. How do load balancers pick a server? 📌 MOST REPEATED

A load balancer sits in front of your servers and decides which one should handle each incoming request. Round robin just takes turns — request 1 goes to server A, request 2 to server B, and so on. Least connections sends each new request to whichever server currently has the fewest active requests, which works better when some requests take longer than others. Consistent hashing always sends the same user to the same server every time, which is useful for logins or caching, since that server already has the user's data ready to go. On top of all this, a good load balancer constantly runs health checks, and stops sending traffic to any server that has crashed or stopped responding.

Q3. How do you design bit.ly?

To build something like bit.ly, you first need a way to turn a long URL into a short, unique code — this can be done with a simple counter that keeps increasing, or a hash function, then converted into letters and numbers so it stays short. You store this mapping of 'short code → long URL' in a database. Since people click on short links far more often than they create new ones, you put a cache like Redis in front of your database — this way, popular links load almost instantly without hitting the database every single time someone clicks.

Q4. How do you stop API spamming?

To stop someone from spamming your API, you can use a token bucket — imagine every user has a bucket of tokens that slowly refills over time, and each request uses up one token; once the bucket is empty, new requests get blocked. This method is nice because it still allows small, natural bursts of traffic. Another approach is the sliding window, which counts how many requests came in during the last few seconds, giving smoother and more even limiting. One important detail: if you have multiple servers, you can't just count requests locally on each one — you need to store the count somewhere shared, like Redis, otherwise a user could sneak around the limit by hitting different servers.

Q5. How do you handle single points of failure?

A single point of failure is any one part of your system that, if it breaks, takes the entire system down with it — usually this is one database or one load balancer that everything else depends on. The fix is to never rely on just one copy of anything: run multiple copies of your important services, set up automatic failover so a backup instantly takes over if the main one dies, and spread your servers across different regions or zones.

A well-designed system should also degrade gracefully when something does fail — meaning some features might slow down or turn off temporarily, but the whole system doesn't crash completely.

Q6. What is a CDN and why use it?

A CDN (Content Delivery Network) is a network of servers spread out across different locations around the world, and each one stores cached copies of your static content — things like images, videos, and JavaScript files. When a user visits your website, they get served from the CDN server closest to them instead of your main server far away, which makes pages load much faster. The one downside is that if you update the original file on your main server, the cached copies sitting on the CDN can take a little time to catch up and show the new version.

DATABASES

Q1. SQL vs NoSQL — when to pick which? 📌 MOST REPEATED

The choice between SQL and NoSQL should depend on your actual data and use case, not just habit or preference. Choose SQL when your data is structured, has clear relationships between different pieces (like customers and their orders), and you need strong consistency — for example in banking, where every transaction must be perfectly accurate at all times. Choose NoSQL when your data structure keeps changing, you have a huge volume of writes, or you need to operate at massive scale — MongoDB works well for flexible documents, Redis or DynamoDB are great for fast, simple lookups, and Cassandra is built specifically to handle very heavy write loads.

Q2. What is sharding? 📌 MOST REPEATED

Sharding means splitting one giant database into smaller pieces, called shards, and spreading them across multiple servers — usually based on some key, like the user's ID. This lets you scale far beyond what a single database server could ever handle on its own. The hard part is twofold: first, moving data around whenever you add or remove shards is complicated and risky; second, any query that needs data from multiple shards at once becomes slow and expensive, since it has to reach out to several servers and then combine the results.

Q3. What is replication?

Replication means keeping multiple copies of the same data on different servers, so you don't depend on just one machine. In a leader-follower setup, all writes go to one main server (the leader), and the other servers (followers) copy that data and can be used to serve read requests — this also gives you scaling for reads plus a built-in backup. There's a trade-off in how replication happens: synchronous replication is safer because the leader waits for followers to confirm they've saved the data too, but this makes writes slower; asynchronous replication is faster since the leader doesn't wait, but if the leader crashes right after a write, that newest piece of data could be lost before it ever reached the followers.

Q4. Why do indexes help, and the catch?

An index is like a shortcut that lets the database find specific rows quickly, instead of scanning through every single row one by one — similar to the index at the back of a book, which lets you jump straight to a

topic instead of reading the whole thing. The catch is that every index you add makes writing data (inserts, updates, deletes) a little slower, because the database now has to update that index as well, on top of updating the actual table. Because of this trade-off, you should only add indexes on columns that you actually filter, sort, or join on often — not on every single column just in case.

Q5. CAP theorem in plain words? 📌 MOST REPEATED

The CAP theorem says that during a network failure (called a 'partition', when parts of your system can't talk to each other), you can only guarantee two out of these three things at the same time: Consistency (every user sees the exact latest data), Availability (the system always responds to requests, even if the data might be a little outdated), and Partition tolerance (the system keeps working even when the network has problems). Since network partitions are going to happen eventually no matter what you do, in practice this really comes down to choosing between Consistency and Availability. For a banking system, you'd pick consistency, because showing the wrong balance is dangerous. For a social media feed, you'd pick availability, because it's completely fine if a post shows up a few seconds late for some users.

Q6. Strong vs eventual consistency?

Strong consistency means that every time you read data, you're guaranteed to get the very latest write that happened — this is essential for something like a bank account balance, where showing outdated numbers could cause real problems. Eventual consistency means that right after a piece of data is updated, different servers might show slightly older versions of it for a short while before they all catch up — but in exchange, you get much better speed and availability. This trade-off is completely fine for things like the number of likes or views on a post, where being off by a few seconds doesn't matter to anyone.

Q7. How to design a schema like a food delivery app?

When designing a schema for something like a food delivery app, start by identifying your main entities: Users, Restaurants, Orders, and Payments, and figure out how they connect to each other. For data that changes often, like menu prices, you normalize it — meaning you store it in one single place and reference it, so an update only needs to happen once. But for data that gets read a lot and should never change after the fact — like the price of an item at the moment someone actually ordered it — you copy or 'freeze' that data directly onto the Order itself. This way, if a restaurant raises its prices tomorrow, it doesn't accidentally change what a customer already paid for an order placed yesterday.

CACHING

Q1. Where can you add a cache?

You can add a cache almost anywhere in your system — in the user's browser, at the CDN layer, in a dedicated caching layer like Redis or Memcached sitting in front of your database, or even inside the database itself. The general rule is simple: cache whatever data is read very often or is expensive to compute, and try to place that cache as close to the user as possible, so the data travels the shortest distance — but always be careful not to show the user outdated information for too long in the process.

Q2. Cache eviction policies? 📌 MOST REPEATED

When a cache gets full, it needs to decide what to throw out to make room for new data — this decision is called an eviction policy. LRU (Least Recently Used) removes whatever hasn't been accessed in the longest time, and it's the most common default because it works well for most real-world usage patterns. LFU (Least Frequently Used) removes whatever has been accessed the fewest number of times overall, which can be smarter for items that are rarely popular in general. FIFO (First In, First Out) simply removes the oldest item, no matter how often it's actually being used right now — it's the simplest to build, but also the weakest, since it might throw out something that's still very popular.

Q3. Write-through vs write-back vs write-around?

Write-through means every write goes to both the cache and the database at the same time — this keeps them perfectly in sync and is very safe, but it makes writes a little slower since you're writing to two places at once. Write-back means the write goes to the cache first and gets saved to the database later — this is much faster, but risky, because if the cache crashes before that data reaches the database, you lose it for good. Write-around skips the cache completely and writes straight to the database — this works well for data that gets written once and rarely read again, since there's no point caching something nobody's going to look at anytime soon.

Q4. Why is cache invalidation hard?

Cache invalidation is considered one of the hardest problems in computing, because there's no perfect way for the system to know exactly when cached data has become stale, or outdated. One common approach is TTL (Time To Live), where cached data automatically expires after a set amount of time — this is simple to set up, but it means the cache could keep serving old data right up until the moment it expires. The other approach is to manually clear or update the cache the instant the real data changes — this is more accurate, but it's also easy to forget a spot or make a mistake, which then causes confusing bugs where some users see stale data and others don't.

Q5. How to prevent a cache stampede? 📌 MOST REPEATED

A cache stampede happens when a very popular cached item expires all at once, and suddenly a huge number of requests all hit the database at the exact same time trying to recompute that same data — this can overload and even crash your database. You can prevent this a few ways: use locking, so only one request is allowed to recalculate the value while every other request simply waits for it to finish; combine duplicate requests asking for the same thing into a single shared request; or randomize the expiry times slightly, so popular items don't all expire at the exact same moment and cause a sudden spike.

DISTRIBUTED SYSTEMS

Q1. What is consistent hashing? 📌 MOST REPEATED

Consistent hashing works by placing both your servers and your pieces of data onto an imaginary circular ring, using a hashing function. Each piece of data is then assigned to the next server found going clockwise on that ring. The big advantage here is that when you add or remove a server, only a small chunk of the data needs to move to a different server — unlike basic hashing, where adding or removing even one server

would force almost all of your data to be reshuffled and moved around, which would be extremely expensive and slow.

Q2. How do message queues work? 📌 MOST REPEATED

A message queue sits in between the part of your system that creates work (the producer) and the part that actually does the work (the consumer), so the two don't need to run at the same speed or even be active at the same time. This smooths out sudden spikes in traffic, since work simply piles up in the queue instead of overwhelming your servers directly. There are different delivery guarantees worth knowing: at-most-once means a message might get lost and never processed at all; at-least-once means a message might get delivered more than once, so your code needs to be able to handle duplicates safely; and exactly-once, where every message is processed exactly one time with no loss and no duplicates, is technically the hardest guarantee to actually achieve in practice.

Q3. What is a distributed lock and when needed?

A distributed lock is used to stop two different servers from touching the same piece of data at the exact same time — for example, making sure two people can't both book the very last seat on a flight at once. These locks are usually built using tools like Redis or Zookeeper. One critical detail: you should always give a lock an expiry time. If you don't, and the process holding the lock crashes unexpectedly, that lock could stay locked forever, freezing that resource for everyone else permanently.

Q4. How does a distributed system elect a leader?

In a distributed system, nodes need to agree on one single 'leader' to handle certain important jobs, so that two different nodes don't end up fighting for control at the same time — this dangerous situation is called 'split-brain'. Leader election is usually done using algorithms like Raft or Paxos, or tools like Zookeeper, which constantly send heartbeats between nodes to check whether the current leader is still alive. If the leader stops responding, the system detects this and automatically triggers a new election to pick a replacement. While there's no leader in place, during that brief moment of election, any writes to the system are typically paused or queued up until a new leader has been chosen.

Q5. What does idempotent mean and why care? 📌 MOST REPEATED

Idempotent means that doing something once, or doing it many times by accident, always gives you the exact same result. For example, 'set the balance to 100' is idempotent — no matter how many times you run it, the balance ends up at 100. But 'add 10 to the balance' is not idempotent, because running it multiple times just keeps adding more and more. This matters a lot in real systems because network failures cause retries all the time — if a payment request fails to get a response and your app automatically retries it, and that action isn't idempotent, you could accidentally charge someone twice. The standard fix is to send a unique idempotency key with every request, so the server can recognize and safely ignore any duplicate attempts.

Q6. How do you design for eventual consistency on purpose?

Designing for eventual consistency on purpose means accepting that data might be slightly out of date across different servers for a short period, in exchange for getting better speed and always staying available

to users. This works well for things like like-counts, view-counts, or recommendations, where it genuinely doesn't matter if the number is a few seconds behind reality. To build this, you typically use asynchronous replication between servers, and you make sure your application and your users are okay with occasionally seeing slightly stale, outdated data for a brief moment.

REAL PRODUCT DESIGNS

Q1. How would you build a WhatsApp-style chat app?

To build a chat app like WhatsApp, you'd use WebSockets, which keep an open, instant, two-way connection between the user's phone and your server, so messages can be delivered in real time without constantly refreshing the page. Messages get stored and organized by conversation, so they can be loaded quickly whenever someone opens a chat. For users who are offline, a queue holds their messages until they come back online and reconnect. For group chats, the same message needs to be sent out to every single member's individual queue. It's also worth mentioning end-to-end encryption, so only the sender and receiver can actually read the messages, and how you'd keep messages in the correct order even if they arrive at different times.

Q2. How would you build a notification system?

The big challenge in a notification system is sending one single event out to potentially millions of users, across multiple channels like push notifications, SMS, and email, all at the same time. The solution is to put the notification jobs into a queue, and then have separate background workers pull jobs from that queue and send them out through whichever channel is appropriate. You also need to handle retries for any sends that fail, make sure you're not spamming any one user with too many notifications at once, and respect each user's individual notification preferences and settings.

Q3. How would you design Swiggy/Zomato? 📌 MOST REPEATED

For an app like Swiggy or Zomato, your main entities are Users, Restaurants, Orders, and Delivery Partners. To find the nearest available rider quickly, you track every delivery partner's live location using a geospatial index, which is a database structure built specifically for fast location-based searches. The order itself should be modeled as a state machine, moving through clear stages — placed, accepted, preparing, picked up, delivered — so the system always knows exactly what state any order is in at any moment. Finally, you'd use a message queue so that other parts of the system, like sending notifications, calculating billing, or logging analytics, can react to order status changes independently, without slowing down the main ordering flow.

Q4. How would you design Uber/Ola? 📌 MOST REPEATED

Designing something like Uber or Ola is similar to food delivery, but it's much more real-time-heavy. You need continuous GPS tracking to know exactly where every driver and every rider is at all times, plus a geospatial index to quickly find the nearest available drivers to a rider's location. The matching algorithm that pairs a rider with a driver should also consider things like the driver's rating and vehicle type. It's also worth mentioning surge pricing, where prices automatically go up when there's high demand and low driver supply in an area, and treating each trip as a state machine, just like an order in food delivery.

Q5. How would you design a payment system? 📌 MOST REPEATED

When designing a payment system, correctness matters far more than raw scale — you absolutely cannot afford to accidentally double-charge someone or lose track of their money. Every single request needs a unique idempotency key attached to it, so that if the same request gets retried due to a network failure, the system recognizes it and doesn't process the payment twice. You'd also use database transactions, or a more advanced pattern called the saga pattern for operations that span multiple services, to make sure a payment either completes fully or rolls back completely — never getting stuck halfway done. Finally, you should keep an unchangeable, append-only transaction log of everything that happens, so you can always audit and trace back exactly what occurred if something ever goes wrong.

Q6. How would you design a news feed like Instagram/Twitter? 📌 MOST REPEATED

When designing a news feed like Instagram or Twitter, the key trade-off to understand is between 'fan-out on write' and 'fan-out on read'. Fan-out on write means that the moment someone posts something, that post gets immediately pushed out and copied into all of their followers' feeds — this makes reading the feed very fast, but it becomes a huge problem for accounts with millions of followers, since one single post would need to be copied millions of times. Fan-out on read means the feed gets built on the spot, right when someone opens the app, by pulling together posts from everyone they follow — this is cheaper when someone posts, but a bit slower when someone actually opens their feed. Most real-world systems use a mix of both: regular users get the fast fan-out-on-write treatment, while celebrity accounts with huge followings use fan-out on read instead.

Q7. How would you design a parking lot system?

A parking lot system design question is less about handling massive scale, and more about testing your ability to design clean, well-organized classes. You'd typically define classes like ParkingLot, Floor, Spot, Vehicle, and Ticket, with each class having one clear, specific job to do. You'd need to handle different vehicle sizes, like motorcycles, cars, and trucks needing different spot types, decide how spots get assigned to incoming vehicles (nearest available spot first, by floor, etc.), and cover important edge cases — like what happens when the parking lot is completely full, and how the pricing model works, whether it's charged hourly or as a flat rate.

Found this useful?

Follow along for more content like this, dropped regularly.

 **Follow @yashuintech**

for more SDE interview & job-prep
content

 **Follow @yashudeveloper**

for AI & vibe coding content